

ZENDS TelecomAI Brand Intelligence System

Complete Master Documentation & Architecture Report

1. PROJECT OVERVIEW

1.1 Purpose

The **ZENDS TelecomAI Brand Intelligence System** is a full-stack AI-powered platform for a fictional telecom company called "ZENDS Communications." It provides two core capabilities:

1. **Sentiment Analysis** — Real-time classification of customer feedback using a hybrid DistilBERT + LLM pipeline.
2. **RAG (Retrieval-Augmented Generation)** — An AI assistant that answers customer queries by retrieving relevant information from corporate policy documents.

1.2 Business Problem

Telecom companies receive massive volumes of unstructured customer feedback. Manually reading, classifying, routing, and responding to these tickets is slow, expensive, and error-prone. This system automates the entire lifecycle: **ingestion** → **classification** → **routing** → **response generation** → **analytics**.

1.3 End-to-End Workflow

1. A customer visits the **Customer Portal** (Streamlit) and submits feedback or a query.
2. Feedback undergoes **Two-Stage Sentiment Analysis**: DistilBERT baseline → Groq LLM refinement (Llama-3.3-70b).
3. The system classifies sentiment (Positive/Negative/Neutral/Mixed) and auto-detects the category (e.g., Network, Billing).
4. For negative/mixed sentiment, a **support ticket** is auto-generated and persisted to `live_feedback.json`.
5. The **RAG pipeline** (LangGraph state machine) retrieves relevant policy documents and generates a grounded AI response.
6. All transactions are **logged to PostgreSQL** for auditing.
7. The **Analyst Dashboard** visualizes real-time KPIs, sentiment trends, complaint heatmaps, and AI-generated recommendations.

1.4 How Sentiment Analysis & RAG Interact

- The Customer UI uses sentiment analysis to classify and route feedback.
 - The RAG system retrieves policy documents relevant to the detected category and generates contextual responses.
 - Both systems feed data into `live_feedback.json`, which the Analyst Dashboard reads for real-time visualization.
 - The RAG pipeline logs all queries/responses to PostgreSQL via the `log_transaction` node.
-

2. TECH STACK

2.1 Frontend

Technology	Role	Why Chosen
Streamlit	UI framework for both Customer Portal and Analyst Dashboard	Rapid prototyping for data apps with Python
Plotly	Interactive charts (donuts, heatmaps, stacked bars, scatter)	Rich, interactive visualizations
Custom CSS (injected via <code>st.markdown</code>)	Premium UI with dark/light modes, glassmorphism, animations	Full design control beyond Streamlit defaults
Google Fonts (IBM Plex Sans, Sora, JetBrains Mono)	Typography	Professional, modern appearance

2.2 Backend

Technology	Role
FastAPI (<code>api.py</code>)	REST API backend — endpoints for <code>/ask</code> , <code>/upload</code> , <code>/register</code> , <code>/login</code> , <code>/documents</code> , <code>/health</code>
Uvicorn	ASGI server running FastAPI
python-multipart	Handles multipart file uploads for document ingestion

2.3 AI/ML

Technology	Role
LangGraph	Orchestrates RAG as a state machine with conditional routing, retry logic, and fallback
LlamaIndex	Document ingestion, chunking, VectorStoreIndex, retrieval
Langchain (<code>RecursiveCharacterTextSplitter</code>)	Context-aware text chunking (1000 chars, 150 overlap)
HuggingFace Transformers	DistilBERT sentiment baseline via <code>pipeline()</code>
sentence-transformers (<code>all-MiniLM-L6-v2</code>)	384-dim embedding model for RAG vector search
Groq API (<code>llama-3.3-70b-versatile</code>)	LLM for sentiment reasoning, RAG generation, and hallucination grading
PyMuPDF (<code>fitz</code>)	PDF text extraction for document ingestion

2.4 Database & Storage

Technology	Role
PostgreSQL (psycpg2-binary)	Primary DB for users and query audit logs (src/db.py)
SQLite	Lightweight fallback user/API-key store (database/db.py)
FAISS (faiss-cpu)	Local vector similarity search index (vector_store.py)
LlamaIndex Storage (storage_v2/)	Persisted docstore, vector store, index store as JSON
JSON file (live_feedback.json)	Real-time message broker between Customer UI and Analyst Dashboard

2.5 Authentication

Technology	Role
passlib[bcrypt]	Password hashing (configured but not yet fully wired)
python-jose[cryptography]	JWT token management (configured for future use)

2.6 Internationalization

File	Role
locales.py	i18n support for English, Español, Français — all UI labels, buttons, placeholders

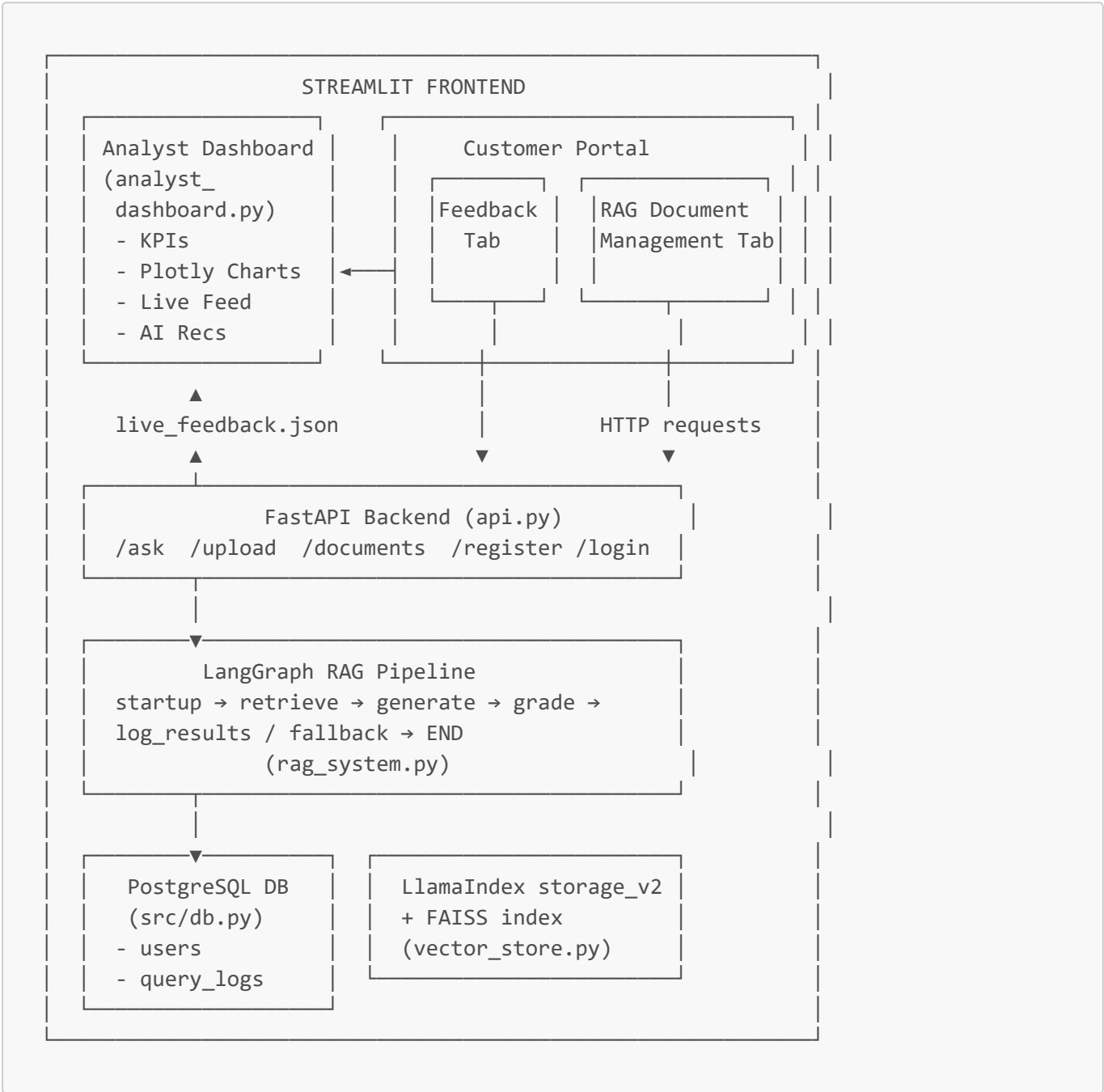
2.7 Key Versions (from [requirements.txt](#))

Package	Purpose
streamlit	Frontend
fastapi, uvicorn[standard]	Backend API
groq	LLM inference (Llama-3.3)
langgraph	RAG state machine
llama-index, llama-index-embeddings-huggingface	RAG indexing & retrieval
sentence-transformers	Embedding generation
faiss-cpu	Vector similarity search
pymupdf	PDF parsing
psycpg2-binary, SQLAlchemy	PostgreSQL
passlib[bcrypt], python-jose[cryptography]	Auth
plotly	Dashboard charts
langchain-text-splitters	Document chunking
pandas, numpy	Data manipulation

Package	Purpose
python-dotenv	Environment config
huggingface_hub	Model downloads

3. ARCHITECTURE ANALYSIS

3.1 High-Level Architecture



3.2 Folder Structure

```
Zends/  
├── .gitignore           # Version control exclusions  
├── analyst_dashboard.py # 729 lines – Analyst Dashboard UI (Plotly
```

```

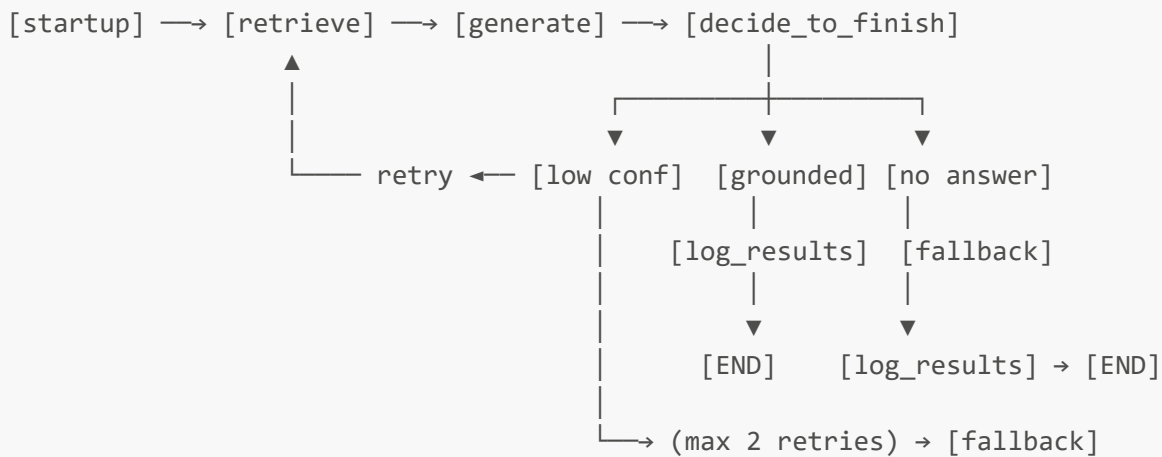
charts, KPIs)
├─ customer_ui.py                # 833 lines – Customer Portal UI (feedback + RAG
management)
├─ merge_ui.py                  # 75 lines – Build script that merged
customer_ui + rag_management
├─ rag_management.py            # 283 lines – Standalone RAG admin UI (now
merged into customer_ui)
├─ project_root/
│   └─ .env                    # Environment secrets (GROQ_API_KEY, Postgres
credentials)
│   └─ api.py                  # 141 lines – FastAPI endpoints
│   └─ streamlit_app.py        # 100 lines – Main entry point, navigation
router
│   └─ locales.py              # 54 lines – i18n translations (EN/ES/FR)
│   └─ requirements.txt        # 22 dependencies
│   └─ telecom_feedback_dataset.csv # 10,000-row synthetic training dataset
│   └─ Zend data.ipynb        # Jupyter notebook – dataset generation, EDA,
DistilBERT training
│   └─ vector_index.faiss      # Persisted FAISS index
│   └─ chunks.pkl              # Persisted chunk metadata
│   └─ src/
│       └─ __init__.py         # Package marker
│       └─ rag_system.py       # 284 lines – LangGraph state machine (core RAG
logic)
│   └─ contract_parser.py      # 118 lines – PDF extraction + text chunking
│   └─ vector_store.py         # 108 lines – FAISS index creation & query
│   └─ embeddings.py           # 63 lines – Embedding utilities (Groq/fallback)
│   └─ db.py                   # 136 lines – PostgreSQL: users, query_logs
│   └─ database/
│       └─ db.py               # 48 lines – SQLite fallback for user management
│   └─ data/
│       └─ fixed_extracted_text.txt # Default RAG knowledge base
│       └─ live_feedback.json     # Real-time feedback store (Customer→Analyst
bridge)
│   └─ uploads/                # User-uploaded documents for RAG
│   └─ storage_v2/              # LlamaIndex persisted index (docstore,
vector_store, etc.)
│   └─ results/                # DistilBERT training checkpoints (500, 1000,
1500 steps)
│   └─ tests/
│       └─ test_api.py         # 33 lines – FastAPI TestClient tests

```

3.3 Entry Points

Entry Point	Command	Purpose
<code>streamlit_app.py</code>	<code>streamlit run streamlit_app.py</code>	Frontend — routes to <code>analyst_dashboard.py</code> or <code>customer_ui.py</code>
<code>api.py</code>	<code>uvicorn api:app --reload</code>	Backend — FastAPI REST API

3.4 RAG Pipeline Flow (LangGraph State Machine)



Nodes:

- **startup_and_ingest** — Loads or creates the LlamaIndex vector index from `storage_v2/`
- **retrieve** — Uses `index.as_retriever(similarity_top_k=5)` to fetch top 5 relevant chunks
- **generate** — Sends retrieved context + user question to Groq LLM (`llama-3.3-70b-versatile`, `temp=0.2`)
- **hallucination_grader_with_score** — Secondary LLM call that scores confidence 0-100 and checks groundedness
- **decide_to_finish** — Conditional router: if grounded & score>60 → log; if "I don't know" → fallback; else retry (max 2)
- **fallback_handler** — Simulates human agent routing ("Routed to US-East Support Queue")
- **log_transaction** — Persists query+response to PostgreSQL `query_logs` table

4. SENTIMENT ANALYSIS SYSTEM

4.1 Architecture: Two-Stage Hybrid Pipeline

Stage 1 — DistilBERT Baseline (in `customer_ui.py:load_sentiment_model()`)

- Model: `distilbert-base-uncased-finetuned-sst-2-english` (HuggingFace `pipeline`)
- Cached via `@st.cache_resource` to avoid reload
- Provides fast initial label (Positive/Negative) and confidence score

Stage 2 — Groq LLM Reasoning (in `customer_ui.py:analyze_feedback()`)

- Model: `llama-3.3-70b-versatile` via Groq API
- Structured JSON output with `response_format={"type": "json_object"}`
- Uses **few-shot prompting** with explicit rules to avoid the "Conjunction Trap"
- Output: `{"sentiment", "category", "confidence", "reasoning"}`
- 4 labels: Positive, Negative, Neutral, Mixed
- 8 categories: Network Connectivity, Billing & Payments, Data Services, Roaming & International, Customer Support, Device & SIM, Plan & Subscription, Voice & Calls

4.2 Custom Training (from `Zend data.ipynb`)

- **Synthetic Dataset:** 10,000 rows generated from templates (3000 Positive, 3000 Negative, 3000 Neutral, 1000 Mixed)
- **Model:** `DistilBertForSequenceClassification` with 4 labels
- **Training:** 3 epochs, batch size 16, max 1500 steps, eval every 500 steps
- **Training Loss:** Dropped from 0.466 (step 50) → 0.00045 (step 500)
- **Eval Loss:** 0.000286 at step 500
- **Checkpoints:** Saved at steps 500, 1000, 1500 in `results/` (~267MB model each)
- **Architecture:** 6 layers, 12 heads, 768 dim, vocab 30522, transformers v5.7.0

4.3 Prompt Engineering Strategy

The LLM prompt includes:

- Role: "Expert Telecom Data Scientist for ZENDS Communications"
- Explicit "Conjunction Trap" avoidance rule
- Instruction to evaluate sentiment **only toward the brand**, ignoring external factors
- Few-shot examples demonstrating correct classification
- DistilBERT baseline result injected as additional context for the LLM

4.4 Full Lifecycle

```
User Input → DistilBERT (fast baseline) → Groq LLM (reasoning + refinement)
→ JSON {sentiment, category, confidence, reasoning}
→ If Negative/Mixed: auto-generate ticket (TCK-XXXXXX)
→ Persist to live_feedback.json
→ Display results + AI response in Customer Portal
→ Analyst Dashboard reads live_feedback.json for real-time KPIs
```

5. RAG SYSTEM

5.1 Document Ingestion Pipeline

1. **Upload:** User uploads PDF/TXT via Customer Portal → FastAPI `POST /upload`
2. **Save:** File saved to `data/uploads/` directory
3. **Extract:** PyMuPDF (`fitz`) extracts raw text from PDFs; `.read_text()` for TXT files
4. **Chunk:** `RecursiveCharacterTextSplitter` (chunk_size=1000, overlap=150, separators: `\n\n, \n, ., , ""`)
5. **Embed:** `all-MiniLM-L6-v2` generates 384-dimensional vectors (LlamaIndex `Settings.chunk_size=256, chunk_overlap=50`)
6. **Store:** LlamaIndex `VectorStoreIndex` persisted to `storage_v2/` (docstore.json, default_vector_store.json, index_store.json)

5.2 Retrieval & Generation

```
Query → startup_and_ingest (load index)
→ retrieve (similarity_top_k=5, cosine similarity)
```

```

→ generate (Groq LLM with system prompt: "Answer using ONLY the provided
context")
→ hallucination_grader (secondary LLM: scores 0-100, checks grounded yes/no)
→ if grounded & score>60: log to DB → END
→ if not grounded & retries<2: loop back to retrieve
→ if retries>=2 or "I don't know": fallback (human agent routing) → log → END

```

5.3 Key Parameters

- **Embedding Model:** `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions)
- **Chunk Size:** 1000 chars (contract_parser), 256 (LlamaIndex Settings)
- **Chunk Overlap:** 150 chars (contract_parser), 50 (LlamaIndex Settings)
- **Top-K Retrieval:** 5
- **LLM Temperature:** 0.2 (generation), 0.0 (grading)
- **Confidence Threshold:** 60%
- **Max Retries:** 2

6. DATABASE & DATA FLOW

6.1 PostgreSQL Schema (`src/db.py`)

```

-- Users table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Query audit log
CREATE TABLE query_logs (
    id SERIAL PRIMARY KEY,
    username TEXT NOT NULL,
    query TEXT NOT NULL,
    response TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

6.2 SQLite Fallback Schema (`database/db.py`)

```

CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    api_key TEXT NOT NULL
);

```


6.3 JSON Data Store (`live_feedback.json`)

```
{
  "id": "TCK-161483",
  "timestamp": "2026-05-16T22:13:39",
  "category": "Network Connectivity",
  "sentiment": "Negative",
  "feedback": "The signal dropped again...",
  "confidence": 100,
  "status": "Pending" // or "Resolved" or "No Action"
}
```

- Written by Customer UI on each feedback submission
- Read by Analyst Dashboard for live feed, Issues Pending/Resolved KPIs
- Capped at 50 most recent entries

6.4 Environment Variables (`.env`)

- `GROQ_API_KEY` — Groq API authentication
- `POSTGRES_HOST`, `POSTGRES_PORT`, `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`

7. API ANALYSIS (`api.py`)

Endpoint	Method	Request	Response	Purpose
<code>/register</code>	POST	<code>{"username", "password"}</code>	201 or 400	User registration
<code>/login</code>	POST	<code>{"username", "password"}</code>	200 or 401	Credential verification
<code>/upload</code>	POST	<code>multipart/form-data (file)</code>	200 or 500	Upload + ingest document into RAG
<code>/ask</code>	POST	<code>{"question", "username"}</code>	<code>{"answer", "confidence", "context", "status"}</code>	RAG query via LangGraph pipeline
<code>/documents</code>	GET	—	<code>{"documents": [{"filename", size_kb}]}</code>	List uploaded documents
<code>/documents/{filename}</code>	DELETE	—	200 or 404/500	Delete doc + rebuild index
<code>/health</code>	GET	—	<code>{"status": "ok"}</code>	Health check

Error Handling: All endpoints use `try/except` with `HTTPException` for proper HTTP error codes. **Document Deletion:** Triggers full index rebuild — removes `storage_v2/`, re-ingests default data, then re-ingests remaining uploads.

8. AI AGENT / AUTOMATION LOGIC

8.1 LangGraph State Machine (Agentic RAG)

The RAG system exhibits **agentic behavior** through its self-correcting state machine:

- **Evaluation Node** (`hallucination_grader_with_score`): A secondary LLM call that audits the generated response against retrieved facts
- **Conditional Routing** (`decide_to_finish`): Dynamically decides whether to log, retry, or fallback based on confidence scores
- **Retry Mechanism**: Up to 2 retrieval retries before falling back to human agent routing
- **Fallback Handler**: Simulates regional routing ("Routed to US-East Support Queue, assigning to available agent")

8.2 Prompt Engineering

- **Generation Prompt**: "You are an expert AI assistant for a telecom company. Answer accurately using ONLY the provided context. If the context does not contain the answer, reply exactly with 'I don't know'."
- **Grading Prompt**: "You are an expert auditor. Compare the GENERATION against the FACTS. Even if different wording or synonyms are used, evaluate if the core meaning is supported. Output JSON with 'score' (0-100) and 'grounded' ('yes'/'no')."
- **Sentiment Prompt**: Includes few-shot examples, explicit "Conjunction Trap" rules, brand-specific evaluation criteria.

8.3 Ticket Auto-Generation

- Tickets (`TCK-XXXXXX`) are auto-generated **only for Negative or Mixed sentiment**
 - Ticket status tracks through `Pending` → `Resolved` lifecycle
 - `mark_resolved()` function in Customer UI updates `live_feedback.json`
 - Analyst Dashboard KPIs show Issues Pending / Issues Resolved counts
-

9. SYSTEM DESIGN EVALUATION

9.1 Strengths

1. **Modularity**: Clean separation — `api.py` (routing), `rag_system.py` (AI logic), `db.py` (storage), Streamlit (UI)
2. **Reliability**: LangGraph hallucination grader ensures enterprise-grade response accuracy
3. **Hybrid AI**: Two-stage sentiment (DistilBERT + LLM) avoids single-model limitations
4. **UX/UI**: Extensive custom CSS creates a premium, professional interface with dark/light modes
5. **i18n**: Multi-language support (English, Spanish, French) via `locales.py`
6. **Full Lifecycle**: Ingestion → Classification → Response → Logging → Visualization

9.2 Areas for Improvement

1. **Vector DB:** FAISS/local storage → migrate to Pinecone, Weaviate, or pgvector for production scale
 2. **Concurrency:** File-based `live_feedback.json` → Redis/Kafka for high-concurrency scenarios
 3. **Security:** Implement JWT token issuance on `/login`, enforce password hashing with bcrypt
 4. **Model Serving:** Isolate DistilBERT inference to a dedicated microservice to prevent UI memory bloat
 5. **Testing:** Expand `test_api.py` (currently only 2 basic tests) to comprehensive integration tests
 6. **CI/CD:** Add deployment pipeline (Docker, cloud hosting)
-

